

FUNCTIONAL PROGRAMMING FOR SCIENTIFIC COMPUTING

Bezlepsy A.D.

Abstract:

Functional programming has increasingly become popular in the field of scientific computing due to its concise and expressive syntax, its reliability, modular design, parallelism and lazy evaluation. In this article, we give a brief introduction to history of functional programming, talk about core concepts of functional programming: data immutability, recursion, lazy evaluation, discuss several key advantages of this paradigm over imperative approach. Overall, we argue that functional programming can provide a powerful tool set for scientists and researchers, enabling them to write efficient, reliable, and scalable code for complex scientific problems.

Keywords:

Functional programming, scientific computing, lazy evaluation, parallelism

INTRODUCTION

When it comes to programming, efficiency, reliability, and scalability are key factors for every scientific researcher. To increase productivity, the choice of programming language and paradigm is critical. Trends in modern scientific computing show that functional programming is favored by researchers for its concise, expressive syntax and high degree of modularity and parallelism.

In this article, we examine the history of functional programming, explain the core concepts of functional programming, and discuss its several advantages over the imperative approach that researchers can benefit from. This article is intended for scientists and researchers interested in improving code efficiency and reliability.

HISTORY OF FUNCTIONAL PROGRAMMING

The origins of functional programming can be traced back to the work of Alonzo Church in the 1930s¹. Church developed the lambda calculus to study the fundamentals of mathematics. Lambda calculus is a formal system for expressing computation as functions, which forms the basis of functional programming.

Functional programming became more mainstream in the 1950s and 60s. John McCarthy, the inventor of the Lisp programming language, introduced the concept of recursion², a key feature of functional programming. Lisp was one of the first functional programming languages to become a great tool for exploring artificial intelligence, and its descendants are still in use today³.

¹ [“Some History of Functional Programming Languages”](#) – D. A. Turner, 2012, p. 1;

² [“History of Lisp”](#) – J. McCarthy, 1979, p.5

³ [“Structure and Interpretation of Computer Programs”](#) – H. Abelson, G.J. Sussman, J.Sussman, p.4 et seq

Functional programming continued to grow in the 70's and 80's. The introduction of lazy evaluation⁴ at that time made it possible to use memory more efficiently. In the 1990s, functional programming started to become important in industry. Companies such as Microsoft and Sun Microsystems began developing functional programming languages for commercial use. This led to the development of such broadly adopted functional languages such as F# and Scala.

An example of a functional programming language used specifically for scientific computing is R. R is a language and environment for statistical computing and graphics. It provides a variety of statistical and graphical techniques and is used in fields such as genetics, finance, and epidemiology.

Another example of a functional programming language used for scientific computing is Julia. It is a fairly new high-level, high-performance programming language. Julia's functional programming paradigm allows writing concise, expressive, and efficient code well suited for scientific computing.

CORE CONCEPTS OF FUNCTIONAL PROGRAMMING

Functional programming is a popular paradigm in computer science that emphasizes the use of functions to solve problems. In this chapter, we will explore some of the core concepts of functional programming, including recursion, data immutability, and lazy evaluation. We will also provide examples of these concepts using expressive syntax of modern programming language Julia⁵.

RECURSION

Recursion is a powerful technique in functional programming that involves defining a function in terms of itself. This allows for the creation of complex algorithms and data structures that would be difficult to implement using iterative methods. Consider the following imperative example of a factorial function in Julia:

```
function factorial_loop(n)
    result = 1
    for i in 1:n
        result = result * i
    end
    return result
end
```

This function calculates the factorial of a given number n using quite an ordinary ever familiar for-loop. Let us see, how can we rewrite it in more functional way.

```
function factorial_recursion(n)
    if n == 0
        return 1
    else
        return n * factorial(n - 1)
    end
end
```

Here instead of iterative loop we see a recursive function call and conditional construction (if-else) to distinct recursive case from the basic one. But Julia's expressiveness allows us to go even further:

```
factorial(x) = x == 0 ? 1 : x * factorial(x-1)
```

⁴ "[Conception, evolution, and application of functional programming languages](#)" - Paul Hudak, ACM Computing Surveys, Vol. 21, No. 3, September 1939, p.385

⁵ [Julia 1.8 official documentation paper](#), 2023, p.977 et seq.

Here we use ternary operator, which is basically a minimalistic conditional construction and declare a function, using a one-line style so keywords “function” and “end” can be omitted. Recursion is a powerful tool in functional programming, which allows to create elegant and efficient algorithms.

DATA IMMUTABILITY

Data immutability refers to the concept of creating data structures that cannot be modified once they are created. Immutable data structures are preferred in functional programming because they are less prone to errors. Data structures are immutable by default in Julia. In scientific computations, where data integrity is crucial, this method may be helpful⁶. Immutable data structures are ideal for parallel programming because they eliminate the possibility of data corruption during multi-core processing (so called “race condition”, an undesirable behavior, when two or more processes read from and write to the same place in memory), making them more effective and thread-safe.

LAZY EVALUATION

Lazy evaluation is a technique in functional programming where expressions are only evaluated when they are needed. This can improve performance by avoiding unnecessary computations. Consider the following example:

```
squares = Iterators.Stateful(x^2 for x in 5:100)
```

Here we have created an iterator, which is to square each number from 5 to 100, but the calculations remain postponed. Hence we temporarily divided creation and computation of an object. So later we can call:

```
popfirst!(squares)
```

which tells Julia to compute the first element of an iterator (25 in our case) while removing it from the queue. If we call it once again, 36 will return. The calculation is performed on demand. This can save computational resources and improve overall performance.

Mentioned concepts can be particularly useful in scientific computing, where time efficiency and data integrity are important consideration.

ADVANTAGES OF FUNCTIONAL PROGRAMMING OVER IMPERATIVE APPROACH

Because of its distinct features and benefits, functional programming has grown in popularity in recent years. The benefits of functional programming over the traditional imperative approach will be discussed in this chapter.

The idea behind functional programming is to break down (reduce) complex problems into smaller, more manageable ones. This is accomplished by utilizing functions, which take inputs and produce outputs. These functions can be combined to create larger programs that can be reused in various contexts⁷. This modular design allows for code reuse, making code maintenance and updating easier.

Contrastingly, control structures like loops and conditional statements are used in imperative programming. They make modularizing code difficult, resulting in code that is difficult to read, maintain, and debug⁸.

Parallelism is an important feature of scientific computing because it enables faster processing of large amounts of data. Because it avoids mutable state and side effects, functional programming lends itself well to parallelism. This makes it easier to reason about the program's behavior and parallelize the code.

⁶ "О функциональном программировании" – Городняя Л. В., “Компьютерные инструменты в образовании, 2021” p. 62

⁷ "Why Functional Programming Matters"– John Hughes, “Research Topics in Functional Programming”, 1990, p.4.

⁸ "Programming Languages for Scientific Computing" - Matthew G. Knepley, 2018, p.5

In contrast, imperative programming relies heavily on mutable state, making parallelization difficult. This is because mutable state can cause race conditions, in which multiple threads access and modify the same data at the same time, resulting in unpredictable results.

CONCLUSION

As we can see, functional programming has several advantages over the traditional imperative approach, making it a valuable tool for both researchers and programmers. Functional programming can help researchers tackle complex problems more effectively and efficiently by enabling modular design, facilitating parallelism, and improving code reliability and maintenance. As the field of scientific computing grows and evolves, functional programming is likely to become an increasingly important tool for researchers across a wide range of disciplines.

REFERENCES

- [1] [“Some History of Functional Programming Languages”](#) – D. A. Turner, 2012, p. 1;
- [2] [“History of Lisp”](#) – J. McCarthy, 1979, p.5;
- [3] ["Structure and Interpretation of Computer Programs"](#) – H. Abelson, G.J. Sussman, J.Sussman, p.4 et seq.;
- [4] ["Conception, evolution, and application of functional programming languages"](#) – Paul Hudak, ACM Computing Surveys, Vol. 21, No. 3, September 1939, p.385;
- [5] [Julia 1.8 official documentation paper](#), 2023, p.977 et seq.
- [6] ["О функциональном программировании"](#) – Городняя Л. В., “Компьютерные инструменты в образовании, 2021” p. 62;
- [7] ["Why Functional Programming Matters"](#)– John Hughes, Research Topics in Functional Programming” ed. D. Turner, Addison-Wesley,1990, p.4;
- [8] ["Programming Languages for Scientific Computing"](#) - Matthew G. Knepley, 2018, p.5.